

LabReSiD26 – Hands-On 12

Davide Ferrara – 518629

1 Esercizio 1: La shell base

In questo esercizio si implemeenta un ciclo *read-fork-exec-wait*: l'obiettivo è leggere un comando da `stdin`, fare `fork()`, eseguire il programma nel figlio con `execvp()` e aspettarne la terminazione nel padre con `waitpid()`.

Prompt e lettura

Il prompt viene stampato con `printf` seguito da `fflush(stdout)`: `printf` usa un buffer *line-buffered* e, poiché "minish> " non termina con `'\n'`, senza `fflush` il testo resterebbe nel buffer e non apparirebbe sullo schermo prima che il programma si blocchi su `fgets()`.

Figlio: `execvp` e padre: `waitpid`

`execvp(argv[0], argv)` sostituisce il processo con il programma richiesto cercandolo nel `PATH`. Se ha successo non ritorna mai: il codice dopo è raggiungibile solo in caso di errore, dove si termina con `exit(127)` (convenzione POSIX per *command not found*). Il padre chiama `waitpid(pid, &status, 0)` e decodifica il risultato con le macro `WIFEXITED/WEXITSTATUS` e `WIFSIGNALED/WTERMSIG`.

```
1 if (pid == 0) {
2     /* execvp non ritorna mai in caso di successo */
3     if (execvp(argv[0], argv) == -1) {
4         perror(argv[0]);
5         exit(127);
6     }
7 } else {
8     /* waitpid() aspetta il figlio specificato */
9     int status;
10    waitpid(pid, &status, 0);
11    if (WIFEXITED(status) && WEXITSTATUS(status))
```

```

12     printf("[Exit %d]\n", WEXITSTATUS(status));
13     if (WIFSIGNALED(status))
14         printf("[Signal %d]\n", WTERMSIG(status));
15 }

```

Listing 1: Figlio ed padre – parti cruciali

1.1 Output atteso

Come si può notare dall'output, la shell è in grado di eseguire **ls**, di fare **echo** e di uscire con **exit code 127**.

```

1 $ ./minish_es1
2 minish> ls -l
3 total 472
4 -rwxr-xr-x 1 dff dff 15784 May 13 crash
5 -rw-r--r-- 1 dff dff 63 May 13 crash.c
6 -rwxr-xr-x 1 dff dff 16600 May 13 minish_es1
7 minish> echo ciao mondo
8 ciao mondo
9 minish> comando_inesistente
10 comando_inesistente: No such file or directory
11 [Exit 127]
12 minish> exit

```

Listing 2: Test Esercizio 1

2 Esercizio 2: Zombie, segnali e cd

L'esercizio estende la shell con tre funzionalità: esecuzione in background con **&**, raccolta asincrona dei figli tramite **SIGCHLD** e il built-in **cd**.

Comandi in background con &

Se l'ultimo token della riga è **&**, si rimuove da **argv**, si stampa il PID del figlio e si torna al loop senza chiamare **waitpid()**.

```

1 /* Il loop e' necessario: i segnali dello stesso tipo
   non si accodano,
2 * un singolo SIGCHLD puo' rappresentare N terminazioni.
   */
3 static void sigchld_handler(int sig) {
4     (void)sig;

```

```

5     while (waitpid(-1, NULL, WNOHANG) > 0)
6         ;
7 }
8
9 ...
10
11 /* In main, prima del loop: */
12 signal(SIGCHLD, &sigchld_handler);
13
14 ...
15
16 /* Rilevazione &: */
17 int bg = argc > 0 && !strcmp(argv[argc - 1], "&");
18 if (bg)
19     argv[--argc] = NULL;
20
21 /* Nel padre: */
22 if (bg) { printf("[1] %d\n", pid); continue; }

```

Listing 3: Rilevazione background e handler SIGCHLD

Raccolta asincrona con SIGCHLD

Quando un figlio in background termina, il kernel manda SIGCHLD al padre. L'handler chiama `waitpid(-1, NULL, WNOHANG)` in un loop per raccogliere tutti i figli terminati, evitando i processi zombie.

Senza handler – zombie accumulati:

```

1 dff    73704    S+    ./minish
2 dff    73731    Z+    [sleep] <defunct>
3 dff    73783    Z+    [sleep] <defunct>
4 ...    (15 zombie)

```

Con handler – nessun zombie:

```

1 dff    72437    S+    ./minish
2 dff    73119    S+    grep -E minish|defunct

```

Built-in cd

`chdir()` modifica la directory corrente del processo che la chiama. Se lo facessimo in un figlio, solo la directory del figlio cambierebbe; la shell rimar-

rebbe nella directory di partenza. Per questo `cd` viene gestito prima della `fork()`, con `continue` per saltarla. Senza argomenti va in `$HOME`.

```
1 if (!strcmp(argv[0], "cd")) {
2     if (argc == 1) {
3         const char *home = getenv("HOME");
4         if (!home) { fprintf(stderr, "cd: HOME non
                    impostata\n"); continue; }
5         if (chdir(home) < 0) perror("cd");
6     } else {
7         if (chdir(argv[1]) < 0) perror("cd");
8     }
9     continue; /* non fare fork */
10 }
```

Listing 4: Built-in `cd`

3 Esercizio 3: La pipe

L'esercizio estende la shell per supportare `cmd1 | cmd2`: i due comandi vengono eseguiti in due processi figli collegati da una pipe del kernel.

Rilevazione di `|` e `split` di `argv[]`

Si scorre `argv[]` finché non si trova `|`: si imposta `argv[i] = NULL` per terminare il vettore sinistro e si salva `argv_right = &argv[i+1]` come puntatore al comando destro.

```
1 char **argv_right = NULL;
2 int i = 0;
3 while (i < argc) {
4     if (!strcmp(argv[i], "|")) {
5         argv[i] = NULL; // Split
6         argv_right = &argv[i + 1]; // Argomenti di
                    destra
7         break;
8     }
9     i++;
10 }
```

Listing 5: Ricerca di `|` e pipeline

Esecuzione della pipeline

Prendendo come esempio `ls | grep .c`: `grep` deve leggere da `stdin` l'output di `ls`. Si creano due processi figli: il primo reindirige `stdout` sul capo di scrittura della pipe (`fd[1]`), il secondo reindirige `stdin` sul capo di lettura (`fd[0]`). Gli indici della pipe rispecchiano la convenzione dei file descriptor standard: `fd[1]` è il capo di scrittura (come `stdout=1`) e `fd[0]` è il capo di lettura (come `stdin=0`).

```
1 pid_t child1 = fork();
2 if (child1 == 0) {
3     dup2(fd[1], STDOUT_FILENO); /* stdout -> pipe */
4     close(fd[0]); close(fd[1]);
5     execvp(argv_left[0], argv_left);
6     exit(127);
7 }
8 pid_t child2 = fork();
9 if (child2 == 0) {
10     dup2(fd[0], STDIN_FILENO); /* stdin <- pipe */
11     close(fd[1]); close(fd[0]);
12     execvp(argv_right[0], argv_right);
13     exit(127);
14 }
15 /* Il padre chiude entrambi i capi: se non chiudesse fd
16    * child2 non riceverebbe mai EOF e rimarrebbe bloccato.
17    */
17 close(fd[0]); close(fd[1]);
18 waitpid(child1, NULL, 0);
19 waitpid(child2, NULL, 0);
```

Listing 6: Fork, redirect e attesa della pipeline

3.1 Output atteso

```
1 minish> ls -l | grep .c
2 crash.c
3 minish.c
4 minish_es1.c
5 minish_es2.c
6 minish> cat /etc/passwd | grep root
7 root:x:0:0::/root:/usr/bin/bash
8 minish> echo uno due tre | wc -w
9 3
10 minish> ps aux | grep minish
11 dff 110138 0.0 0.0 2752 1792 pts/3 S+ 23:18 0:00
    ./minish
12 dff 111198 0.0 0.0 6676 4212 pts/3 S+ 23:24 0:00
    grep minish
```

Listing 7: Test Esercizio 3 – pipe